为什么要自己写切分器

现成工具能用，但它切出来的东西，和读者要核对的东西不是同一个单位

22.4%	80.7%	874	569
现成方案带条款号	自研方案带条款号	现成切出条数	自研切出条数

对照实验：LangChain RecursiveCharacterTextSplitter vs 本项目 splitter.py

同一批 PDF、同一套抽取文本，只换切分方式

一、先给结论

现成工具在「切得动」这件事上没有任何问题：能切、长度可控、有重叠、代码一行就写完。它的问题不在质量，在于引用单位错位——字符窗口产出的是一段没有名字的文字，而读者要核对的是「第十二条，第几页」。

• 引用无法核对

拿到「第 12 页第 3
段」，读者没法确认你引用的是不是他理解的那一段。按条款切，引用能写成「DOC011 第二章 第 3.4 条 · 第 24 页」，翻过去就能对上。

• 结构信息丢失

现成方案不产生章节路径与条款号。合同纠纷常发生在「总则」与「专用条款」冲突的地方，丢掉章节归属，检索命中后连「这是哪一章的规定」都说不清。

• 句子被截断

按句号切会把「第X条」从中间劈开，一条规定分到两个
chunk，两边都不完整。重叠只能缓解，不能消除——重叠部分还会被重复计分。

所以自研不是因为「现成的不好」，而是因为产品的核心承诺是「答得可被核对」，核对的最小单位是条款，切分就必须以条款为单位。这是需求倒逼的，不是技术偏好。

二、这个结论是比出来的，不是想出来的

为了避免「我觉得」，做了一个可复现的对照实验：同一批 PDF、同一套 PyMuPDF 抽取文本，只有切分方式不同。现成方案取最常见的那种写法——LangChain 的 RecursiveCharacterTextSplitter，500 字符窗口、100 字符重叠、分隔符优先级为 段落 > 换行 > 句号 > 分号 > 逗号。

```
uv run --with langchain-text-splitters python scripts/compare_splitter.py
```

测评五个维度：切出条数、长度分布、是否以条款编号开头（决定引用能不能被核对）、是否以句末标点结尾（决定语义是否完整）、页码是否落在正确页上。

总量对比

指标	现成方案	自研方案	差在哪
切出条数	874	569	现成切得更碎，因为它是按长度切
中位长度	464 字符	239 字符	自研服从条款本身的长度，长短不一
P90 长度	495 字符	1188 字符	现成长度很均匀；自研有长条款，这是它的代价
最长	499 字符	1200 字符	自研设了 1200 上限，超长条款按标点切开
以条款编号开头	22.4%	80.7%	差 3.6 倍，这是核心差距
以句末标点结尾	32.4%	70.1%	现成方案有三分之二的 chunk 停在半句话上

值得说清楚的是第四行：长度均匀性上现成方案更好，因为它本来就是按长度切的。自研的代价就是长度分布更散（86–1200 字符），长条款对向量化不友好。这个代价我认，因为用它可以换来结构信息；但它是真实存在的，不该在汇报里藏起来。

三、逐份对照

十份可解析文档，差距在第一列就看得出来：现成方案的「带编号」比例在 4%–57%
之间浮动，自研方案稳定在
62%–97%。浮动本身就是问题——它意味着同一套参数在不同文档上表现完全不同。

文档	字符	现成条数	现成中位	现成带编号	自研条数	自研中位	自研带编号
DOC002	3,991	11	451	18%	14	216	93%
DOC003	61,314	173	470	10%	107	222	72%
DOC004	8,238	27	458	4%	14	438	93%
DOC005	39,309	112	462	11%	122	86	97%
DOC008	72,412	210	464	57%	97	538	62%
DOC009	30,273	85	461	12%	80	178	90%
DOC010	5,289	15	460	13%	15	248	87%
DOC011	72,419	209	463	9%	101	507	77%
DOC012	5,543	15	472	33%	9	366	78%
DOC014	5,762	17	459	53%	10	388	80%

DOC005 那一行也值得看：自研切出 122 条、中位只有 86 字符，比现成的 112 条更多。原因是这份招标文件的内容都在「1、2、3」这种短条目里，条款级切分自然更碎。这不是缺陷，是它本来的结构——只要引用能对上，碎一点没关系。

四、同一段原文，两种切法

宁国市垃圾中转站项目合同第五条（付款方式）。下面是两种方案各自切出来的东西，原文一字未改。

现成方案：一条 chunk 里装了两条规定的残片

「制造商及中标投标人负责，其余由采购人组织验收，必要时可以委托质检部门（或第三方专业机构）或有关……项目的质量保证金在质保期(1年)满后的10个工作日内一次性付清（不计利息）。第六条：售后服务」

开头没有条款号——因为它从「第五条」的中间开始；结尾停在「第六条」的标题上——第六条的内容在下一个 chunk 里。这一条既不能回答「第五条讲了什么」，也不能回答「第六条讲了什么」。

自研方案：一条完整的第五条

DOC002-C0006 | clause_no = 第五条 | 第 5 页 | 138 字符
「第五条：付款方式：货到采购人指定安装点安装完毕并经验收合格且经第三方水质检测机构检测合格后付合同价款的70%；设备使用6……」

读者拿到这条引用，能直接翻到第 5
页找第五条，逐字核对。这就是差别：前者是一段文字，后者是一个可以被指认的条款。

另一个更隐蔽的问题：页码残留进了正文

现成方案的 chunk49 开头是「—\n19\n—」，这是 PDF 页脚的页码，被当成了正文。自研方案在清洗阶段就把跨页重复的短行丢掉了，因此不会有这种东西。这类噪声会进入向量，也会进入模型的上下文。

五、自研的代价，一并说清

代价	具体是什么	怎么控制
代码量	text.py 234 行 + splitter.py 380 行 + models.py 106 行，共 720 行	其中约一半在处理异常分支，这部分是边界情况的沉淀，不是重复劳动
长度不均	条款短则 86 字符，长则 1200 字符，对向量化不友好	设 1200 字符硬上限并按标点切开，片段继承父条款编号
边界情况多	目录页、裸标题、跨页、表格、只有标题没正文、编号层级错乱	每一条都写进了单元测试，22 项测试里 13 项在管这些
依赖自己的判据	层级选择靠「中位长度 60–700」这类经验阈值	阈值写成常量并在 README 说明理由；D10 做消融时会重新校准

六、什么情况下不该自研

这一点必须写清楚，否则「自研」就成了万能答案。判断标准只有一条：引用是否需要落到读者可核对的最小单位上。

场景	该用什么	原因
读报告、答泛化问题	现成切分器	没有核对诉求，长度均匀反而更好
笔记、网页、博客检索	现成切分器	原文没有稳定结构，硬切结构是自找麻烦
合同、招标、法规、条款	条款级切分	引用必须能指认到「第X条」，这是刚需
需要按章节归纳	版面分析工具（Docling、unstructured）+ 结构切分	先拿结构，再切；不要从纯文本里猜结构

七、面试里怎么答（30 秒）

「试过 LangChain 的递归切分器，能切，但切出来的东西没法用来做引用——它切 500 字符一段，一段里可能装着两条规定的残片，开头还没有条款号。我做过对照：同样十份文件，它切 874 条，只有 22% 以条款编号开头；按条款切是 569 条，81% 带编号。差别不在切分质量，在于读者能不能凭引用去核对。我的产品承诺就是『答得可被核对』，所以切分必须按条款来。代价是 720 行代码和更散的长度分布，这部分我认。」

附录 术语中英对照

正文里出现过的英文词，按主题分组；产品名与通用缩写保留原名，右侧给出中文与解释。

英文	中文	一句话解释
切分相关		
chunk	切块 / 片段	把长文档切成可检索的小块，每块称为一个 chunk
chunk_size	切块长度	单个块包含的字符或词元数
overlap	重叠	相邻两块共享的尾部内容，避免句子被切断后两边都缺
separator	分隔符	切分时优先在哪些字符处断开，如段落、句号
splitter	切分器	执行切块的程序
clause	条款	合同中最小的可引用单元，如「第五条」
clause_id	条款编号	本项目给每条条款分配的唯一标识，如 DOC002-C0006
parent_clause_id	父条款编号	超长条款被切开后，片段指向原条款的编号
chapter_path	章节路径	条款所属的章、节层级，用于引用时说明出处
citation	引用	答案所依据的原文位置，含文件、条款号、页码
解析与数据		
PDF	便携文档格式	常见的文档格式；文本型 PDF 可直接抽取文字，扫描件不能
OCR	光学字符识别	把图片里的文字识别成文本，扫描件需要它才能检索
header / footer	页眉 / 页脚	每页重复出现的短行，抽取时必须剔除，否则污染正文
TOC	目录	Table of Contents；目录行含点线与页码，会被误认成章节标题
JSONL	按行存储的 JSON	每行一个 JSON 对象，适合流式读写
manifest	清单文件	记录一批产物元信息的汇总文件
工程与度量		
token	词元	模型处理文本的最小单位
golden / ground truth	标注答案 / 真值	人工标注的正确答案，用于评测
baseline	基线	改动前的对照结果，用来证明改动是否真的有效
diff	差异	版本控制里两次提交之间的变化
commit	提交	把改动记录进版本库的一次操作
uv / ruff / pytest	——	Python 的依赖管理、代码检查、测试工具

chunk、token、baseline

这类词在中文资料里译法不统一（分块/切块/切片，词元/token），本文统一用左列写法；产品与工具名保留原名。